Model Card: Group-35-AV-Category-C

Model Summary

This is a **3-model Transformer Cross-Encoder Ensemble** with length-and-length-gap-aware bucketed weighting and a long text model for authorship verification. It determines whether two texts were written by the same person by encoding both texts jointly through a RoBERTa cross-encoder and combining predictions from three complementary model variants via a structured bucketed ensemble. A fourth long text model is blended in exclusively for long and extra-long text pairs to mitigate the effect of truncation.

Model Description

The final system is a **two-layer inference pipeline**:

Layer 1: Base Ensemble (3 Models)

Three RoBERTa-based cross-encoders, each fine-tuned with a different training continuation strategy, are combined using a **bucketed weighted average**. The three models are:

1. **Model 1 (`transformer_roberta_20epochs`):** The anchor model, trained from the pretrained `roberta-base` weights for 20 epochs with accuracy-based model selection. This model uses swap augmentation during training and symmetric inference (averaging predictions over both pair orderings) at test time.
2. **Model 2 (`transformer_roberta_hard_negative_precision`):** Initialised from Model 1's best checkpoint and continued for 3 epochs with **hard-negative mining** and **focal loss**. Hard-negative mining identifies different-author pairs with high lexical overlap (token Jaccard ≥ 0.35) or matching URL/email/number patterns, and upweights their loss by 2.5×. Focal loss (γ=1.5) down-weights the loss on already-confident predictions, directing gradient updates towards harder, more ambiguous examples.
3. **Model 3 (`transformer_roberta_symmetry`):** Also initialised from Model 1's best checkpoint and continued for 3 epochs with a **symmetric consistency regularisation** term (λ=0.1). During training, both orderings `(text_1, text_2)` and `(text_2, text_1)` are passed through the model, and a penalty proportional to the squared difference between the two output probabilities is added to the loss: `λ × (P_forward − P_reverse)²`. This encourages order-invariant predictions.

Each input pair is assigned to one of **12 buckets** based on total word count (short/medium/long/extra long) × absolute per-text length difference (balanced/diff/verydiff). Each bucket has its own independently optimised weight vector over the 3 models and its own classification threshold, both tuned on the development set via a simplex grid search. This means different models are trusted more in different regions of the input space, for example, the symmetry model dominates on medium-length balanced pairs (weight 0.98), while the hard-negative model dominates on medium pairs with an imbalanced length (weight 0.96).

Layer 2: Long Text Model

A fourth RoBERTa model (`transformer_roberta_long_text`), trained exclusively on long and extra-long text pairs with an extended maximum sequence length of 384 tokens vs. 256 for the base models. It is initialised from Model 2's checkpoint and trained with the same hard-negative mining and focal loss configuration. For any pair that falls into a `long_*` or `xlong_*` bucket, the base ensemble probability is blended with the long text model's probability using a bucket-specific mixing coefficient α:

```
P_final = (1 − α) × P_base_ensemble + α × P_longtext
```

Both α and the classification threshold are optimised independently per bucket on the development set.

Cross-Encoder Architecture

All four models share the same cross-encoder formulation:

1. **Input:** The two texts are packed into a single sequence separated by the RoBERTa separator token: `<s> text_1 </s></s> text_2 </s>`. This allows full bidirectional token-level attention between both texts.
2. **Backbone:** `roberta-base` (125M parameters) loaded via Hugging Face `AutoModelForSequenceClassification` with `num_labels=2`.
3. **Output:** A 2-class softmax over `[different_author, same_author]`. The probability of class 1 (same author) is the model's confidence score.
4. **Symmetric inference:** At test time, both orderings are passed through the model and their probabilities are averaged, reducing the effect of the order on the prediction.

Pair Feature Extraction

The system computes pair-level features used for bucket assignment and hard-negative identification, including:

- Token Jaccard similarity and character n-gram Jaccard similarity (3-gram, 5-gram)
- Absolute length difference and length ratio
- Punctuation, uppercase, and digit ratio differences
- Repeated character and repeated punctuation ratios
- URL, email, and number co-occurrence indicators
- Placeholder match counts between processed texts

Developers

Kanav Gupta, Xiao Ma, and Yangsong Zhou

Model Details

- **Model Type:** Supervised (Transformer Cross-Encoder Ensemble)
- **Model Architecture:** RoBERTa Cross-Encoder with bucketed ensemble + long text model
- **Language:** English
- **Base Model:** `roberta-base` (Hugging Face)
- **Base Model Repo:** https://huggingface.co/FacebookAI/roberta-base
- **Base Model Papers:**
  - RoBERTa: A Robustly Optimized BERT Pretraining Approach (https://arxiv.org/abs/1907.11692)

Training Data

The model was trained exclusively on the COMP34812 Authorship Verification dataset (approximately 27,000 text pairs). Swap augmentation was applied during training — both `(text_1, text_2)` and `(text_2, text_1)` orderings are used as separate examples, doubling the effective training data, since comparing text A vs text B is the same as comparing text B vs text A.

Training Procedure

Pre-processing

All four models in the final system use **raw-text preservation**: no lowercasing, no URL/email/number normalisation (`lowercase=False`, `normalize_urls=False`,

`normalize_emails=False`, `normalize_numbers=False`). This preserves stylometric features such as capitalisation, URL formatting, and number usage that are informative for authorship verification. Tokenisation for model input uses the RoBERTa byte-pair encoding tokenizer. Tokenisation for bucket assignment uses whitespace splitting to compute word counts and length differences.

Training Process

The training follows a **multi-stage continuation** approach:

```
Stage 1:  Train Model 1 from pretrained roberta-base (20 epochs, accuracy selection)

Stage 2a: Using Model 1's best checkpoint, train Model 2 (hard-negative mining + foca
Stage 2b: Using Model 1's best checkpoint, train Model 3 (symmetric consistency λ=0.1

Stage 3:  Using Model 2's best checkpoint, train the Long Text Model (long/xlong pair

Stage 4:  Simplex grid search over dev set to find per-bucket ensemble weights and th

Stage 5:  Per-bucket α blending search to mix the long text model into the base ensem
```

All training uses AdamW with weight decay, linear warmup followed by linear decay, gradient accumulation (effective batch size 32), gradient clipping (max norm 1.0), and FP16 mixed-precision training on GPU. Early stopping is applied based on the selection metric.

Hyperparameters

Model 1 (`transformer_roberta_20epochs`):

Hyperparameter	Value
Pretrained backbone	`roberta-base`
Max sequence length	256
Epochs	20
Batch size	8 (effective 32 with gradient accumulation ×4)
Learning rate	2e-5
Weight decay	0.01

Hyperparameter	Value
Warmup ratio	0.1
Selection metric	Accuracy
Early stopping patience	3
Swap augmentation	Yes
Symmetric inference	Yes
Preprocessing	Raw text (no normalisation)
Seed	13

Model 2 (`transformer_roberta_hard_negative_precision`):

Hyperparameter	Value
Initialisation	Model 1 best checkpoint
Epochs	3 (continuation)
Learning rate	8e-6
Weight decay	0.01
Warmup ratio	0.05
Selection metric	F1
Early stopping patience	2
Hard-negative mining	Enabled
Hard-negative overlap threshold	0.35 (token Jaccard)
Hard-negative weight	2.5×
Hard-negative placeholder weight	2.0×
Focal loss γ	1.5
Swap augmentation	Yes
Symmetric inference	Yes

Model 3 (`transformer_roberta_symmetry`):

Hyperparameter	Value
Initialisation	Model 1 best checkpoint
Epochs	3 (continuation)
Learning rate	6e-6
Weight decay	0.01
Warmup ratio	0.05
Selection metric	F1
Early stopping patience	2
Symmetric consistency λ	0.1
Swap augmentation	Yes
Symmetric inference	Yes

Long Text Model (`transformer_roberta_long_text`):

Hyperparameter	Value
Initialisation	Model 2 best checkpoint
Max sequence length	384 (extended)
Epochs	3
Batch size	4 (effective 32 with gradient accumulation ×8)
Learning rate	6e-6
Weight decay	0.01
Warmup ratio	0.05
Selection metric	F1
Early stopping patience	2
Hard-negative mining	Enabled (inherited from Model 2)
Focal loss γ	1.5 (inherited from Model 2)
Training data	Only `long` and `xlong` pairs

Hyperparameter	Value
Swap augmentation	Yes
Symmetric inference	Yes
Seed	23

Bucketing Configuration

Primary dimension — total word count:

Bucket	Condition
`short`	total_words ≤ 120
`medium`	121 ≤ total_words ≤ 200
`long`	201 ≤ total_words ≤ 300
`xlong`	total_words > 300

Secondary dimension — absolute per-text length difference:

Bucket	Condition
`balanced`	abs_len_diff ≤ 20
`diff`	21 ≤ abs_len_diff ≤ 60
`verydiff`	abs_len_diff > 60

Speeds, Sizes, and Times

- Overall training time (4 models): approximately 3–4 hours on a single T4 GPU (Google Colab)
- Inference time (full ensemble): approximately 10–15 minutes on a single T4 GPU for ~6,000 pairs
- Model size per checkpoint: ~500 MB (model weights + tokenizer)
- Total inference bundle size (4 models, zipped): ~2 GB

Evaluation

Testing Data

All evaluation metrics are reported on the provided development set (`dev.csv`, approximately 6,000 pairs), which was also used for bucket weight optimisation, threshold tuning, and long text model blending search.

Testing Metrics

- F1-score (Macro)
- Accuracy
- Precision & Recall
- Weighted Macro Precision, Recall, F1
- Matthews Correlation Coefficient (MCC)
- ROC AUC
- Equal Error Rate (EER)
- Log Loss
- Brier Score

Results

On the development set, the model achieved:

Classification Metrics:

- **Accuracy:** 0.8298
- **Macro Precision:** 0.8399
- **Macro Recall:** 0.8281
- **Macro F1:** 0.8280
- **Weighted Macro Precision:** 0.8388
- **Weighted Macro Recall:** 0.8298
- **Weighted Macro F1:** 0.8283
- **Matthews Correlation Coefficient:** 0.6678

Probabilistic Metrics:

- **ROC AUC:** 0.8579
- **Equal Error Rate (EER):** 0.2251
- **Log Loss:** 0.5624
- **Brier Score:** 0.1904

Technical Specifications

Hardware Requirements

- RAM: at least 16 GB
- Storage: at least 5 GB for inference bundles
- GPU: CUDA-compatible GPU (T4 or better recommended)

Software Requirements

- Python 3.x
- PyTorch
- Hugging Face Transformers
- scikit-learn
- pandas
- numpy
- matplotlib, seaborn (for evaluation plots)

Bias, Risks, and Limitations

The model was trained exclusively on the COMP34812 training dataset. Because of this, there are a few limitations:

1. The final system relies on an **ensemble of four models**, not a single model, increasing the training and inference cost.
2. Bucket-specific weights and thresholds were **tuned on the development set**, so there is a risk of overfitting to the dev distribution and generalisation to the hidden test set is not guaranteed.
3. The 256-token (or 384-token for the long text model) input window means that **very long texts are still truncated**, and potentially informative content at the end of long documents is lost.
4. **Short text pairs** are one of the hardest categories to predict as there is less stylometric information available to make a clear judgment.
5. The cross-encoder may conflate **topical similarity with stylistic similarity**: pairs discussing the same subject by different authors may be misclassified as same-author.
6. The model is limited to **English text** and may struggle on examples with code-switching.
7. The model may not **generalise across domains**: it has only seen the provided training distribution and may not transfer well to text from domains with different stylistic conventions.
8. The dataset does not contain examples of authors deliberately trying to imitate another's style, so the model may be fooled by copycat behaviour.

Ethical Considerations

Authorship verification raises important ethical concerns in real-world deployment:

- **Privacy risks.** Analysing personal writing style can reveal identity information even when the author intends to remain anonymous.
- **False positives in high-stakes settings.** Incorrectly attributing authorship could have serious consequences in legal, academic integrity, or journalistic contexts.
- **Potential for misuse.** The technology could be misused for surveillance, profiling, or censorship.
- **Bias.** The model may perform differently across writing styles associated with different demographics, languages, or cultural backgrounds, leading to unfair outcomes.

This system should be treated as an **academic project** and should not be deployed as a production forensic tool without rigorous additional validation, fairness auditing, and ethical review.

Additional Information

Why Raw Text Preservation?

Unlike normalised-text approaches explored earlier in development, the final system preserves the original surface form of all texts. Authorship signals often live in formatting choices — capitalisation patterns, number formatting, URL inclusions — that are destroyed by normalisation. In experiments, raw-text models consistently outperformed their normalised counterparts on the development set.

Why Continuation Training Instead of Separate Models?

All three base models share Model 1 as a common initialisation. This continuation approach is faster than training from scratch, requiring only 3 additional epochs per variant, and it produces models that are diverse in their error patterns (each continuation strategy pushes the model in a different direction) while retaining the strong baseline capabilities of the anchor model.

Why Bucketed Ensembling?

A single global set of ensemble weights underperforms because different models behave differently depending on text length and length asymmetry. The hard-negative model is most useful on pairs where lexical overlap is misleading, while the symmetry model excels on balanced-length pairs. The bucketed design acts as a lightweight mixture-of-experts, routing each pair to the optimal weight combination for its length profile.